Optimizing Gaussian Elimination on the ASUS Tinker Board 2

Introduction

In this study, we will explore performance optimization techniques for a simple Gaussian elimination program written in C++. We will be looking at pivotless Gaussian elimination implementations to focus on performance optimizations rather than numerical stability. Gaussian elimination is a core algorithm in linear algebra used to solve systems of equations, compute matrix inverses, and perform LU decomposition. Although simple to implement, the initial version suffers from poor cache locality and does not take advantage of modern CPU features.

This study will focus on performance on the ASUS Tinker Board 2, an ARM-based device powered by a RockChip RK3399 SoC with a big.LITTLE architecture (dual Cortex-A72 + quad Cortex-A53) and NEON vector instructions. Because the RK3399 is a highly modular "Raspberry Pi-like" device and has limited thermal headroom and memory bandwidth compared to a desktop-level CPU, optimization choices significantly affect performance.

I implemented and benchmarked multiple Gaussian elimination versions, progressively applying:

- Cache Tiling
- Loop Transposition
- Thread-level Parallelism (OpenMP)
- NEON Vectorization
- Minor Software Design Improvements

Each version was benchmarked, profiled, and compared to understand how optimizations with hardware in mind improve runtime.

Problem Statement

Given an *n x n* dense matrix *A*, perform Gaussian elimination to convert *A* into upper triangular form. The classical implementation for this is:

```
// baseline_gaussian_elimination.cpp
```

```cpp
// This is our simple Gaussian elimination program with
upper-triangularization and row-major storage.

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>
#include <cmath>

using Clock = std::chrono::high_resolution_clock;

// Row-major matrix: rows are stored contiguously in memory
struct Matrix {
    int n;
    std::vector<double> data;
    explicit Matrix(int n_) : n(n_), data(static_cast<size_t>(n_) * n_) {}

    inline double& operator()(int i, int j) { return
data[static_cast<size_t>(i) * n + j]; }
    inline const double& operator()(int i, int j) const { return
data[static_cast<size_t>(i) * n + j]; }
};

static Matrix make_random_matrix(int n, uint64_t seed) {
    Matrix A(n);
    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> dist(-1.0, 1.0);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A(i, j) = dist(rng);
        }
    }

    // Make diagonal less likely to be tiny/zero (helps avoid dividing by
zero in baseline).
```

```
    // Still a simple algorithm: so no pivoting; this just stabilizes
random input generation.
    for (int i = 0; i < n; ++i) {
        A(i, i) += (A(i, i) >= 0.0 ? 5.0 : -5.0);
    }
    return A;
}

// Baseline Gaussian elimination.
// Converts A into upper triangular form in-place.
static void gaussian_elimination_baseline(Matrix& A) {
    const int n = A.n;

    for (int k = 0; k < n - 1; ++k) {
        const double pivot = A(k, k);

        // If pivot is 0, This version will fail, we do not handle that
case

        for (int i = k + 1; i < n; ++i) {
            const double factor = A(i, k) / pivot;

            // Start at j = k
            for (int j = k; j < n; ++j) {
                A(i, j) -= factor * A(k, j);
            }
        }
    }
}

// Quick checksum so the compiler can't optimize away work.
static double checksum(const Matrix& A) {
    double s = 0.0;
    for (double x : A.data) s += x;
    return s;
}

static double time_one_run_ms(const Matrix& input) {
    Matrix A = input; // copy so every run uses the same data
    const auto t1 = Clock::now();
```

```
    gaussian_elimination_baseline(A);
    const auto t2 = Clock::now();

    // Prevent dead-code elimination
    volatile double sink = checksum(A);
    (void)sink;

    std::chrono::duration<double, std::milli> dt = t2 - t1;
    return dt.count();
}

int main(int argc, char** argv) {
    int n = 512;
    if (argc >= 2) n = std::atoi(argv[1]);
    if (n <= 0) {
        std::cerr << "Usage: " << argv[0] << " [n]\n";
        return 1;
    }

    // We want all versions to use identical matrices; implemented w/seed
    const uint64_t seed = 0xC0FFEEULL;
    Matrix input = make_random_matrix(n, seed);

    const int runs = 10;
    double total_ms = 0.0;
    double best_ms = 1e300;
    double worst_ms = 0.0;

    for (int r = 0; r < runs; ++r) {
        const double ms = time_one_run_ms(input);
        total_ms += ms;
        best_ms = std::min(best_ms, ms);
        worst_ms = std::max(worst_ms, ms);
    }

    const double avg_ms = total_ms / runs;

    std::cout << "Baseline Gaussian Elimination (no pivoting)\n";
    std::cout << "n = " << n << ", runs = " << runs << "\n";
    std::cout << std::fixed << std::setprecision(3);
```

```
    std::cout << "avg = " << avg_ms << " ms, best = " << best_ms
              << " ms, worst = " << worst_ms << " ms\n";

    return 0;
}
```

The algorithm here is O(n^3) and memory-bound for large matrices. On small embedded platforms, suboptimal memory patterns can cause dramatic slowdowns.

For this experiment I have used matrices of the following sizes:

- n = 256
- n = 512
- n = 1024

These sizes were small enough to run repeatedly on the Tinker Board 2 but large enough to exhibit optimization improvements

All benchmarks were compiled using:

g++ -03 -march-armv8-a+simd -fopenmp

Profiling was performed using perf-tools, gprof, and ARM Streamline.

Experimental Methodology for Benchmarking and Profiling

Time was measured using “std::chrono::high_reolution_clock,” I took timestamp t1 before running Gaussian elimination, ran the algorithm, and took timestamp t2, computed (t2 - t1) and averaged over 10 runs.

To reduce input variability, the same randomly generated matrices (seeded with fixed seeds) were used across all versions of the program.

My experiments were run exclusively on the ASUS Tinker Board 2 (4GB RAM) running the Debian-based TinkerOS.

Profiling data (memory stalls, cache miss ratio, and vectorization opportunity reports) guided my optimization decisions.

Baseline Implementation

The baseline version is a transcription of a textbook Gaussian elimination algorithm with row-major storage. Here is what profiling revealed:

- Approximately ~70% of time spent in the innermost loop
- The cache miss rate for level 1 cache is around ~22%
- No vectorization detected by the compiler
- Only 1 core utilized

Runtime results (in milliseconds):

Version	256 x 256	512 x 512	1024 x 1024
Baseline	118 ms	945 ms	7760 ms

These numbers will serve as the baseline for measuring the impact of our subsequent optimizations.

Optimization Technique 1: Loop Transposition

Motivation:

The baseline stores rows contiguously, but Gaussian elimination performs many column-based accesses, which in turn causes:

- Poor spatial locality
- And many cache-line reloads (when the CPU has to fetch a chunk of data that isn't readily available nearby)

By transposing matrix A, elimination accesses become more cache-friendly.

Results:

Version	256 x 256	512 x 512	1024 x 1024

Transposed	91 ms	731 ms	5810 ms

This gives us about a 25% improvement, consistent with improved locality.

Optimization Technique 2: Cache Tiling

Motivation:

The RK3399's L1 cache is quite small (32 KB per core). This means that standard elimination repeatedly touches entire rows, which in turn, causes the cache to overflow.

Tiling splits up the j-loop into blocks sized to fit in L1, such as 32 x 32 tiles.

Results:

Version	256 x 256	512 x 512	1024 x 1024
Tiled	62 ms	498 ms	3822 ms

Tiling gives us a 35-45% speedup over the transposed version.

Profiling showed L1 cache miss ratio drop from 22% down to 8%.

Optimization Technique 3: Parallelization using OpenMP

Gaussian elimination has an opportunity naturally for parallelization. Each iteration: i = k+1…n-1 can be updated independently.

On our RK3399, we can assign:

- 2 large cores for main computational work
- 4 smaller cores optionally, when thermal throttling is not an issue.

Results using all 6 cores:

Version	256 x 256	512 x 512	1024 x 1024
Tiled + Parallel	26 ms	203 ms	1441 ms

Our speedups are approximately x2.4-x3.0 over the tiled-only version.

Optimization Technique 4: ARM NEON Vectorization

ARM NEON supports 128-bit SIMD (four floats or two doubles). Here is the logic for Gaussian elimination's innermost loop:

A[i][j] -= factor * A[k][j]

This means that: we pick a pivot row *k*, and for each row *i* below it, we eliminate the pivot column entry by subtracting a scaled version of the pivot row. "Factor" here denotes how much of row *k* we need to subtract from row *i*)

The innermost loop can be vectorized using:

- Compiler auto-vectorization
- And explicit NEON intrinsics (vld1q_f32, vmlsq_f32, etc.) "Intrinsics" being functions that look like C code but directly map to a specific CPU instruction.

Results with intrinsics:

Version	256 x 256	512 x 512	1024 x 1024
Tiled + Parallelized + NEON	19 ms	154 ms	1098 ms

This is a 10-20% improvement on top of threading and tiling.

Profiling indicates SIMD utilization at around ~85% efficiency for large sizes.

Software Design Improvements

We also compared structural differences in implementation by testing whether code structure alone affected performance.

To test this, I created two versions of the program:

- A modular version using separate matrix classes and header files
- And a header-only inline version that allows for aggressive inlining

Results:

Version	512 x 512
Modular	158 ms
Header-only / inlined	154 ms

When the modular version of the program was compared with the header-only version, we found that the header-only version ran slightly faster (154 ms vs 158 ms for a 512x512 matrix), which indicates that reduced function-call overhead provides a very modest benefit, compared to algorithmic or cache-level optimizations.

Final Comparison of All Versions

Version	256 x 256	512 x 512	1024 x 1024
Baseline	118 ms	945 ms	7760 ms

+ Transposition	91 ms	731 ms	5810 ms
+ Tiling	62 ms	498 ms	3822 ms
+ Tiling + Parallelization	26 ms	203 ms	1441 ms
+ Tiling + Parallelization + NEON	19 ms	154 ms	1098 ms

Final improvements relative to baseline:

- 6x faster for 1024x1024 matrices
- Approximately ~5x faster for 512x512 matrices

The largest gains came from tiling and parallelization, which makes sense considering the CPU architecture's limited cache and multi-core design.

Conclusion

The main goal of this project was not to change the underlying Gaussian elimination algorithm, but to simply understand how optimizations with hardware in mind affect the performance of a fixed O(n^3) computation on an ARM-based system that is resource-constrained. What this study ultimately measured was how efficiently the RK3399 executes the same mathematical work under different memory access patterns, levels of parallelism, and instruction-level utilization.

It is important to note that, the reported runtimes do not reflect improvements in algorithmic complexity or numerical accuracy. Every version performs the same number of floating-point operations and produces the same upper-triangular matrix, The improvements come entirely from reducing wasted CPU cycles, specifically cache misses, memory stalls, underutilized cores, and unused SIMD lanes. In other words, this project measured how close each implementation came to the hardware's performance ceiling rather than, how "fast" Gaussian elimination is in the abstract.

These results clearly show that, on the ASUS Tinker Board 2, Gaussian elimination is memory-bound before it is compute-bound. Early optimizations such as loop transposition and cache tiling produced large gains by improving both spatial and temporal locality, dramatically lowering L1 cache miss rates. Once memory behavior was improved, parallelization with OpenMP delivered the largest single performance jump by exploiting the RK3399's multi-core big.LITTLE architecture. Only after these higher-level bottlenecks were addressed did NEON

vectorization provide additional gains, confirming that SIMD is most effective when the data pipeline is already well-fed.

Overall, the final optimized implementation achieved up to a 6x speedup over the baseline for large matrices, despite the fact that it uses the same algorithm and input data. This demonstrates a key lesson of performance engineering: the vast majority of real-world speedups come from data movement and concurrency, not from clever math or micro-optimizations. On small embedded systems especially, understanding cache sizes, memory bandwidth, and core layout matters more than raw instruction throughput.

This project reinforced the idea that optimization is an iterative, measurement-driven process. Assessment and profiling guided every major decision, and each optimization revealed the next limiting factor. The results highlights how performance emerges from the interaction between software structure and hardware design, and why writing “correct” code is only the first step towards writing fast code.